PROJET E6 – BTS SIO SLAM 2

Développement d'une application Web sécurisée de gestion de livres et de commandes

Sujet pédagogique – Deuxième année – Épreuve E6

Architecture technique cible

Navigateur / Postman	API REST Express (HTTPS)	Middlewares JWT + rôles	Contrôleurs + modèles Sequelize	Base MySQL
Requête HTTPS + Authorization: Bearer TOKEN	Routes /api/*	Vérifie le token puis le rôle (ADMIN, GESTIONNAIRE, CLIENT)	Logique métier, validation, associations	Tables, clés étrangères, contraintes

Figure 1 – Architecture technique cible

1. Intitulé du projet

- Développement d'une application Web sécurisée de gestion de livres et de commandes.

2. Contexte professionnel

- LibriNova est une librairie en ligne indépendante spécialisée dans la vente de livres neufs, de bandes dessinées et de papeterie culturelle.
- L'entreprise utilise actuellement plusieurs fichiers Excel pour gérer les livres, genres, éditeurs, clients, commandes et utilisateurs. Cette organisation entraîne des risques de doublons, des difficultés de recherche, un manque de contrôle des données et une gestion insuffisamment sécurisée des comptes.
- La direction de LibriNova souhaite mettre en place une application Web centralisée, accessible via une interface sécurisée et exposant des API REST afin de permettre à d'autres applications de communiquer avec le système.

3. Mission confiée au développeur

- Concevoir et développer une application permettant de gérer les utilisateurs, genres, éditeurs, livres, clients, commandes et lignes de commande, ainsi que le stock et des indicateurs de pilotage.
- L'application devra respecter une architecture MVC, utiliser Node.js et Express côté serveur, MySQL pour la persistance et Sequelize comme outil de mapping objet-relationnel.

© 2026 Caplogy. All rights reserved.

4. Technologies imposées

- Node.js – Express.js – MySQL – Sequelize ORM – architecture MVC – API REST – HTML/CSS/Bootstrap/EJS ou frontend JavaScript – JWT – bcrypt – HTTPS/TLS – Postman – Git/GitLab.

5. Objectifs pédagogiques

- Analyser un besoin et produire un modèle de données relationnel.
- Construire un MCD puis un MLD et une base MySQL normalisée.
- Mettre en œuvre les clés primaires, clés étrangères et contraintes d'intégrité.
- Développer un backend Node.js/Express structuré en MVC.
- Créer des modèles Sequelize et leurs associations.
- Développer des API REST utilisant différents verbes et codes HTTP.
- Mettre en œuvre l'authentification JWT, le hachage bcrypt et la gestion des rôles.
- Protéger l'application avec HTTPS, validation des entrées, variables d'environnement et contrôle d'accès.

6. Conception de la base de données

- **Les tables minimales sont :** USER, CLIENT, GENRE, EDITEUR, LIVRE, COMMANDE et LIGNE_COMMANDE.
- **USER :** id_user PK, nom, prenom, email UNIQUE, password, role, date_creation.
- **CLIENT :** id_client PK, nom, prenom, email, telephone, adresse, ville, code_postal, date_creation.
- **GENRE :** id_genre PK, nom, description.
- **EDITEUR :** id_editeur PK, raison_sociale, email, telephone, adresse, ville.
- **LIVRE :** id_livre PK, isbn UNIQUE, titre, auteur, description, prix, stock, seuil_alerte, nb_pages, id_genre FK, id_editeur FK, date_creation.
- **COMMANDE :** id_commande PK, date_commande, statut, montant_total, id_client FK.
- **LIGNE_COMMANDE :** id_ligne PK, quantite, prix_unitaire, id_commande FK, id_livre FK.

Table	Clé primaire	Clés étrangères
USER	id_user	id_client (compte CLIENT, facultatif)
CLIENT	id_client	–
GENRE	id_genre	–
EDITEUR	id_editeur	–
LIVRE	id_livre	id_genre, id_editeur
COMMANDE	id_commande	id_client
LIGNE_COMMANDE	id_ligne	id_commande, id_livre

Figure 2 – Schéma relationnel simplifié à compléter par l'étudiant

© 2026 Caplogy. All rights reserved.

7. Cardinalités obligatoires

- **GENRE 1,N LIVRE :** un(e) genre contient plusieurs livres et un(e) livre appartient à un(e) genre.
- **EDITEUR 1,N LIVRE :** un(e) éditeur peut fournir plusieurs livres.
- **CLIENT 1,N COMMANDE :** un client peut effectuer plusieurs commandes.
- **COMMANDE N,N LIVRE :** une commande contient plusieurs livres et un(e) livre peut apparaître dans plusieurs commandes. Cette relation doit être transformée par la table associative LIGNE_COMMANDE.

8. Contraintes MySQL

- Chaque table doit avoir une clé primaire et les relations doivent utiliser des clés étrangères.
- Les emails et isbn doivent être uniques lorsque cela est pertinent.
- Les champs obligatoires doivent être NOT NULL.
- Le prix doit être supérieur à zéro, le stock supérieur ou égal à zéro et la quantité commandée strictement supérieure à zéro.
- L'étudiant devra justifier les comportements ON DELETE / ON UPDATE choisis : RESTRICT, CASCADE ou SET NULL selon les relations.

9. Sequelize et architecture MVC

- L'application devra séparer au minimum : config, models, controllers, routes, middlewares, views et fichiers publics.
- Les modèles Sequelize devront représenter les tables et leurs relations. Les contrôleurs ne devront pas concentrer toute la logique de l'application.
- **Associations attendues :** Genre.hasMany(Book) / Book.belongsTo(Genre), Publisher.hasMany(Book) / Book.belongsTo(Publisher), Client.hasMany(Order) / Order.belongsTo(Client), ainsi que la relation N:N Order.belongsToMany(Book) / Book.belongsToMany(Order) via OrderLine.

10. API REST obligatoires

- **Authentification :** POST /api/auth/register, POST /api/auth/login, GET /api/auth/profile.
- **Livres :** GET /api/books, GET /api/books/:id, POST /api/books, PUT /api/books/:id, DELETE /api/books/:id.
- **Genres :** GET /api/genres, GET /api/genres/:id, POST /api/genres, PUT /api/genres/:id, DELETE /api/genres/:id.
- **Éditeurs :** GET /api/publishers, GET /api/publishers/:id, POST /api/publishers, PUT /api/publishers/:id, DELETE /api/publishers/:id.
- **Commandes :** GET /api/orders, GET /api/orders/:id, POST /api/orders, PUT /api/orders/:id/status, DELETE /api/orders/:id.

© 2026 Caplogy. All rights reserved.

- **API métier :** GET /api/books/out-of-stock, GET /api/books/low-stock, GET /api/genres/:id/books, GET /api/clients/:id/orders et GET /api/dashboard/statistics.

11. Authentification et autorisation

- Les mots de passe ne doivent jamais être stockés en clair. L'inscription doit utiliser bcrypt.hash() et la connexion bcrypt.compare().
- Après authentification, l'application doit générer un JWT. Le client transmet ensuite le token avec Authorization: Bearer TOKEN.
- Les routes sensibles doivent être protégées par un middleware JWT. Un second middleware devra contrôler les rôles ADMIN, GESTIONNAIRE et CLIENT.
- **ADMIN :** gestion complète. GESTIONNAIRE : livres, stock et commandes. CLIENT : consultation des livres, profil et propres commandes.

12. Sécurité applicative

- Mettre en place HTTPS/TLS pour les échanges.
- Utiliser des variables d'environnement pour les secrets, identifiants et paramètres sensibles.
- Ne jamais versionner le fichier .env.
- Valider les données reçues par les API et refuser les valeurs incohérentes.
- Utiliser des codes HTTP adaptés : 200, 201, 204, 400, 401, 403, 404, 409 et 500.
- Prévoir une gestion centralisée des erreurs et éviter d'exposer des informations sensibles dans les réponses.

13. Interface Web minimale

- **Page de connexion :** email et mot de passe.
- **Dashboard :** nombre de livres, clients, commandes et chiffre d'affaires.
- **Page livres :** liste, recherche, filtre, ajout, modification et suppression selon les droits.
- **Page détail livre :** isbn, nom, description, prix, stock, genre et éditeur.
- **Page commandes :** numéro, date, client, montant, statut et détail de la commande.

14. Tests et validation

- Les API devront être testées avec Postman.
- L'étudiant devra fournir une collection Postman couvrant les cas nominaux et les erreurs : authentification, accès sans token, rôle insuffisant, ressource inexistante, données invalides et conflits.
- Des tests devront également vérifier les contraintes de la base et les principales règles métier, notamment la gestion du stock lors d'une commande.

15. Livrables attendus

- Analyse du besoin et cahier des charges.

© 2026 Caplogy. All rights reserved.

- MCD et MLD justifiés.
- Script SQL de création et de peuplement de la base.
- Projet Node.js/Express complet respectant MVC.
- Modèles Sequelize et associations.
- Documentation des API REST.
- Collection Postman.
- Procédure d'installation et de configuration.
- Documentation de sécurité expliquant JWT, bcrypt, HTTPS, rôles et validation.
- Dépôt Git/GitLab avec historique des principales évolutions.
- Présentation et démonstration de l'application.

16. Contraintes de réalisation

- L'application doit fonctionner avec MySQL.
- Les accès à la base doivent passer par Sequelize pour les opérations courantes.
- Le code doit être organisé selon MVC.
- Les API doivent respecter les principes REST et utiliser des codes HTTP cohérents.
- Les routes sensibles doivent être protégées.
- Les secrets ne doivent pas être écrits en dur dans le code.
- Les mots de passe doivent être hachés avec bcrypt.
- Le protocole HTTPS doit être configuré dans l'environnement de développement.
- Le projet devra comporter au minimum quatre types d'opérations API : GET, POST, PUT/PATCH et DELETE.

17. Travail demandé à l'étudiant

- 1. Analyser le contexte et identifier les acteurs.
- 2. Recenser les fonctionnalités et règles métier.
- 3. Réaliser le MCD puis le MLD.
- 4. Créer la base MySQL avec ses contraintes.
- 5. Initialiser le projet Node.js et Express.
- 6. Mettre en place Sequelize et les modèles.
- 7. Développer les associations et les API.
- 8. Développer l'authentification JWT et bcrypt.
- 9. Mettre en œuvre la gestion des rôles.
- 10. Configurer HTTPS.
- 11. Développer l'interface Web.
- 12. Tester les API avec Postman.
- 13. Documenter les choix techniques et de sécurité.
- 14. Préparer une démonstration de l'application dans le cadre de l'épreuve E6.

© 2026 Caplogy. All rights reserved.

18. Critères de réussite du projet

- Fonctionnement global de l'application.
- Qualité et cohérence de la conception relationnelle.
- Respect des cardinalités et de l'intégrité référentielle.
- Qualité de l'architecture MVC et séparation des responsabilités.
- Pertinence des API REST et des codes HTTP.
- Authentification et autorisation correctement mises en œuvre.
- Sécurisation des mots de passe avec bcrypt.
- Protection des routes par JWT.
- Utilisation effective de HTTPS et protection des secrets.
- Qualité des tests, de la documentation et de la démonstration.

Annexe – Exemple de flux d'authentification

1. L'utilisateur saisit son email et son mot de passe. 2. Le serveur recherche le compte. 3. bcrypt.compare() vérifie le mot de passe. 4. Un JWT est généré. 5. Le client transmet le token dans Authorization. 6. Le middleware JWT vérifie le token avant l'accès aux ressources protégées.

© 2026 Caplogy. All rights reserved.